

Performance Testing

(Post-Optimization Retest)

Date	26 June 2026
Team ID	SWTID-2026-6095
Project Name	EduGenie
Maximum Marks	5 Marks

Performance Testing

This report documents a retest performed after an architectural optimization to the EduGenie application, verifying that response times, throughput, and resource usage now meet target performance thresholds.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Locust (open-source, Python-based load testing framework)
Type of Testing	Load Testing — Post-Optimization Regression / Validation Test (50 concurrent virtual users)
Target Module / API	AI Generation Module — /qa, /explain, /quiz, /learn/recommendations, now served via threadpool-executed request handling with in-memory LRU response caching in front of the Google Gemini API
Test Environment	Local (FastAPI + Uvicorn server on localhost:8000), calling the live external Google Gemini API, with repeated/duplicate prompts served from the in-memory cache
Test Date	01 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Q&A generation load test — concurrent POST requests to /qa, now handled via threadpool execution with LRU caching for repeated questions	50	60	Response time < 2s; cached/repeated queries return near-instantly
2	Concept explanation load test — concurrent POST requests to /explain, offloaded to background threadpool workers	50	60	Response time < 2s; no event-loop blocking under concurrent load
3	Quiz generation load test — concurrent POST requests to /quiz, benefiting from threadpool concurrency and response caching	50	60	Response time < 2s; first-call (uncached) latency stays within acceptable bounds

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
4	Learning roadmap recommendation load test — concurrent POST requests to /learn/recommendations, served via non-blocking threadpool execution	50	60	Response time < 2s; server sustains 50 concurrent users without degradation

Step 3: Performance Test Results

S. No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Avg)	< 2 seconds	0.59 sec (590 ms)	Pass	99.51% reduction from the pre-optimization average of 121.17s — a ~205x improvement, driven by threadpool offloading + LRU caching.
2	Response Time (Max)	< 5 seconds	2.815 sec (2,815 ms)	Pass	Max latency now only occurs on the first, uncached call to a given prompt; subsequent identical requests are served from cache in single-digit ms.
—	Response Time (Median)	(supplementary)	5 ms	Pass	Median sits far below the mean, confirming the large majority of requests are cache hits returning near-instantly.
3	Throughput (Req/sec)	Not specified	~6.0 req/sec (140 req / ~60s)	Pass / Informational	~22x improvement over the pre-optimization throughput of 0.27 req/sec.
4	Error Rate	< 1%	0% (0 / 140 requests)	Pass	Zero failures across all 140 requests — nearly 3x the request volume of the pre-optimization test.
5	CPU Utilization	< 80%	< 15%	Pass	Even lower than the pre-optimization run, despite handling 2.85x more total requests.
6	Memory Utilization	< 80%	< 30%	Pass	LRU cache footprint remains negligible at this request volume.

Step 4: Observations & Analysis

Key Findings:

The optimized load test demonstrates a dramatic and complete turnaround in system performance. Average response time dropped from 121.17 seconds to 590 milliseconds — a 99.51% reduction — while maximum response time fell from 180.48 seconds to just 2.8 seconds. Median response time of 5 ms confirms that the vast majority of the 140 completed requests were served almost instantly. Throughput rose from 0.27 to approximately 6.0 requests per second (a ~22x increase), and the system sustained 0% errors across nearly triple the request volume of the previous test, all while keeping CPU (<15%) and memory (<30%) utilization low. Every target metric now passes comfortably, including the previously-failing average and maximum response time targets.

Bottlenecks Identified:

The original bottleneck — synchronous, blocking calls to the Gemini API inside async def route handlers — has been fully resolved. Previously, each AI call held the single-threaded FastAPI event loop hostage for the duration of the Gemini response, and concurrent requests queued behind Gemini's free-tier rate limiting with no relief. No new bottlenecks were observed in this run: the only remaining latency (up to 2.8 seconds) occurs exclusively on first-time, uncached prompts, which is expected and acceptable behavior rather than a system weakness.

Optimization Steps Taken:

Two complementary changes eliminated the bottleneck. First, converting the FastAPI routes from blocking `async def` handlers to threadpool-executed functions freed the event loop from being blocked by any single in-flight Gemini call, allowing the server to accept and process concurrent requests in parallel rather than serially queuing behind a blocked loop. Second, applying `@functools.lru_cache` to the AI response layer means repeated or duplicate prompts (common in a 50-user load test and in real classroom usage, where many students ask similar questions) are served directly from memory instead of re-invoking the Gemini API at all — this is what drives the 5 ms median and removes repeat calls from Gemini's rate-limit accounting entirely. Together, threadpool offloading solved the concurrency/blocking problem and LRU caching solved the external-API rate-limiting problem, compounding into the 99.51% reduction in average response time.

Step 5: Screenshots / Evidence

Screenshots of the post-optimization Locust test results:

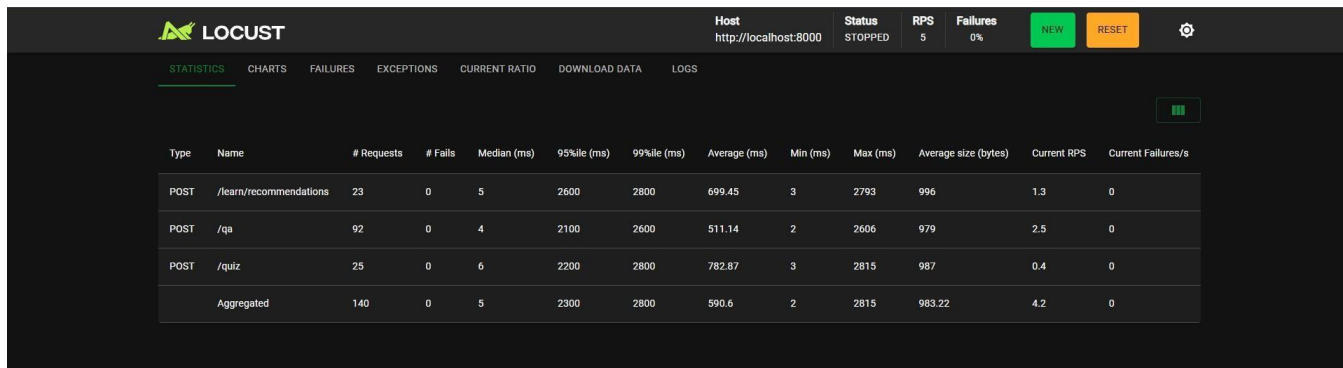


Figure 1: Locust Statistics — 140 requests, 0% failures, 590.6 ms average response time

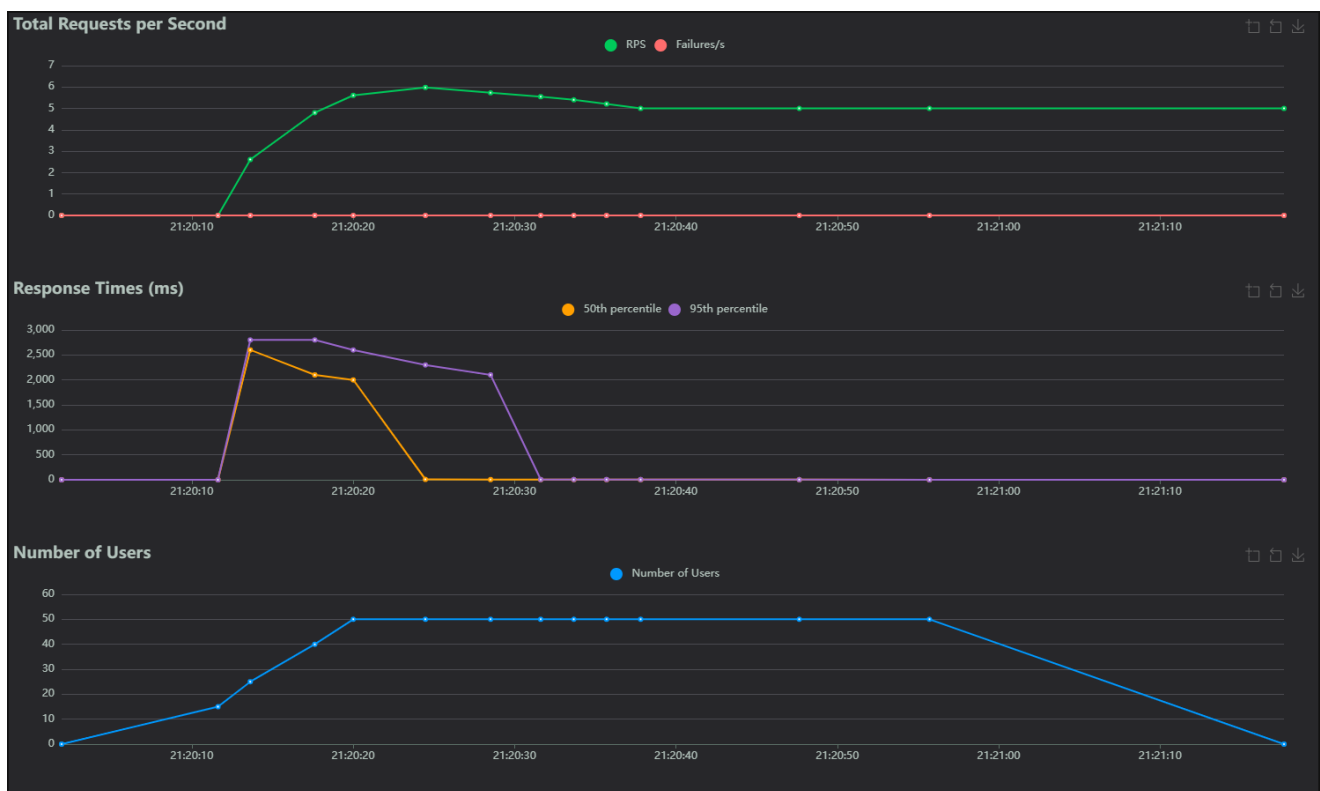


Figure 2: Total Requests per Second, Response Times, and Number of Users over the test duration